

Programming Techniques

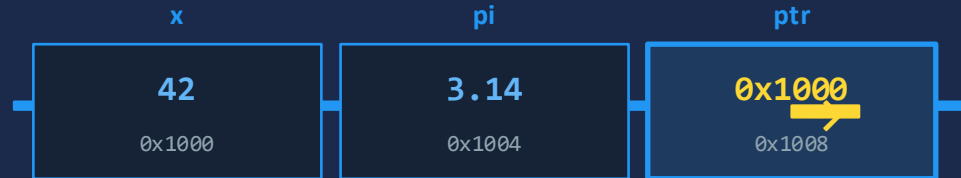
Week 6 — Pointers & Understanding Memory

Memory & Addresses

Pointer Basics

Pointer to Pointer

Pointers & Structs



Kalinga Gunawardhana

Master of Information Technology | Sabaragamuwa University of Sri Lanka

Today's Learning Journey



 All practicals are in **Python only** — C is shown on concept slides for reference and comparison.

Concept slides

~2 hrs

Practical exercises

~60 min

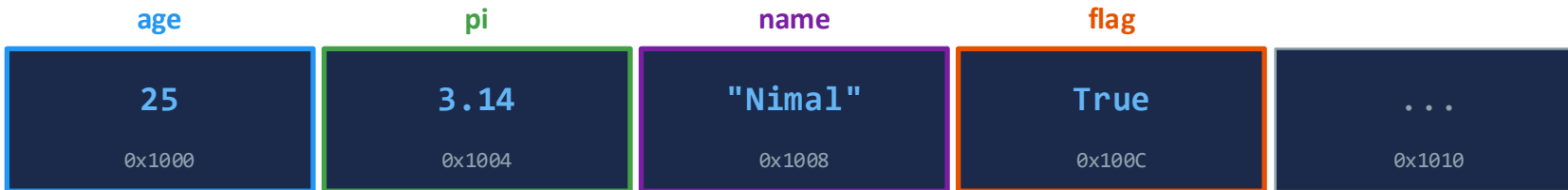
Challenge + wrap-up

~15 min

How Variables Live in Memory

Every variable has two things — a **value** (what it stores) and an **address** (where it lives in RAM).

Memory (RAM) — Each cell has an address:



The Street Analogy

RAM is like a street — every house has a unique address. The family living inside is the value. Python lets you find the address with `id()`.

Python gives you the address

```
age = 25
print(id(age)) # e.g. 140234561298736
```

Value

What is stored — 25, 3.14, "Nimal"

Address

Where it lives — memory location (id)

Seeing Memory Addresses — Python vs C

Python (primary language)

```
x = 42
name = "Nimal"
gpa = 3.8

# id() returns the memory address
print(x)          # 42
print(id(x))      # 140234561298736

print(name)
print(id(name))   # different address

print(gpa)
print(id(gpa))    # different address
```

C equivalent (reference only)

```
int x = 42;
char name[] = "Nimal";
float gpa = 3.8;

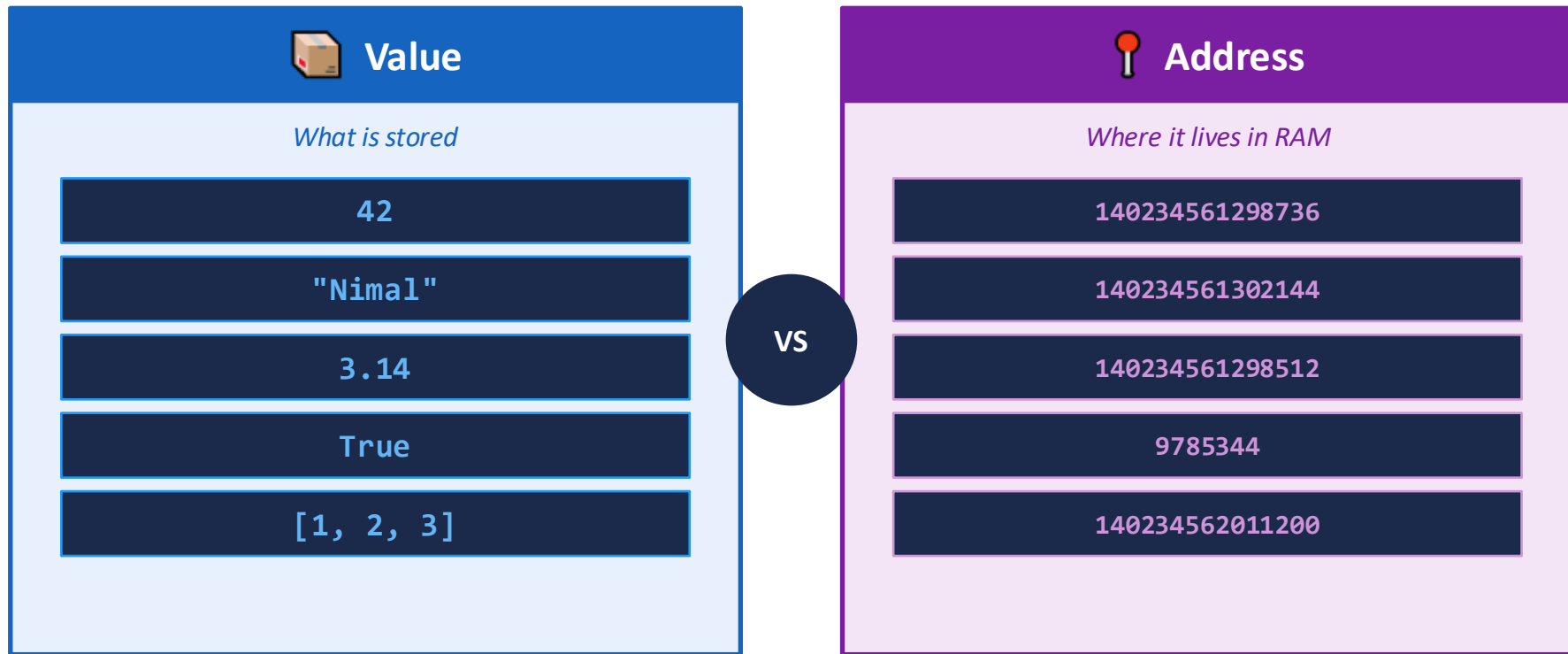
// & gives the address
printf("%d", x);          // 42
printf("%p", &x);        // 0x7fff...

printf("%s", name);
printf("%p", &name);     // different

printf("%f", gpa);
printf("%p", &gpa);     // different
```

Key difference: Python uses `id()` · C uses `&x` — both return the same thing: the memory address where the variable lives in RAM.

Value vs Address — Two Things Every Variable Has



A pointer stores the **address** — not the value itself. It tells you **where to find** the value in memory.



Practical 1 — Exploring Memory in Python



10 min

1 Declare 3 variables: age = 25, name = "Nimal", gpa = 3.8

2 Print each value and its id() — note the addresses

3 Run the script twice — do the addresses change each run? Why?

4 Try: b = age — print id(age) and id(b) — are they the same?

```
age = 25
name = "Nimal"
gpa = 3.8
```

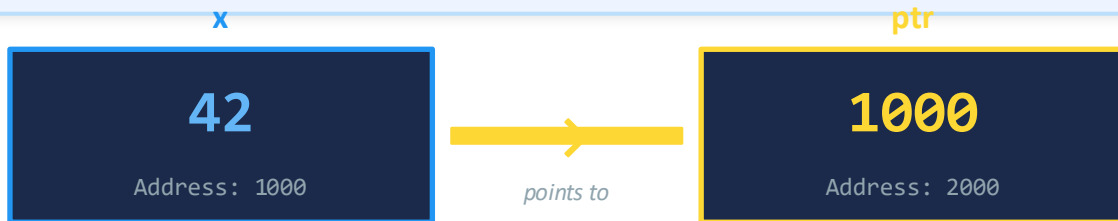
```
print(age, id(age))
print(name, id(name))
print(gpa, id(gpa))
```

```
b = age
print(id(age) == id(b))    # True
or False?
```

Expected insight: b = age makes both variables point to the same object — so id(age) == id(b) is **True**. Addresses change each run because the OS assigns memory dynamically.

What is a Pointer?

A **pointer** is a variable that stores the **memory address** of another variable — it points to where the data lives.



ptr stores the address of x (1000), not the value 42



Regular Variable

Stores a value directly — age = 25



Pointer Variable

Stores an address — ptr = id(x) → 1000



In Python, every variable is already a reference (pointer) under the hood — when you write `x = 42`, Python creates the object 42 in memory, then x stores the address of that object.

How Python Handles Pointers — Python vs C

Python (primary)

```
x = 42
ptr = x          # ptr references same object

print(id(x))     # e.g. 140234561298736
print(id(ptr))   # same address!

# Advanced: read value at an address
import ctypes
val = ctypes.cast(
    id(x), ctypes.py_object
).value
print(val)       # 42
```

C (reference only)

```
int x = 42;
int *ptr = &x;    // explicit pointer

printf("%p", ptr); // address of x
printf("%d", *ptr); // 42 – dereference

// *ptr = dereference:
// "go to that address
// and read the value"
```

Python: all variables are references automatically. **C:** pointers are explicit — you declare them with ***** and get addresses with **&**

The Three Pointer Concepts

Address-of

Give me the address where this variable lives in memory

Python

```
print(id(x))    # 140234...
```

```
printf("%p", &x); // 0x7fff...
```

Pointer Variable

A variable that holds an address — declared with * in C, implicit in Python

Python

```
ptr = x          # stores address of x
```

```
int *ptr = &x; // explicit pointer
```

Dereference

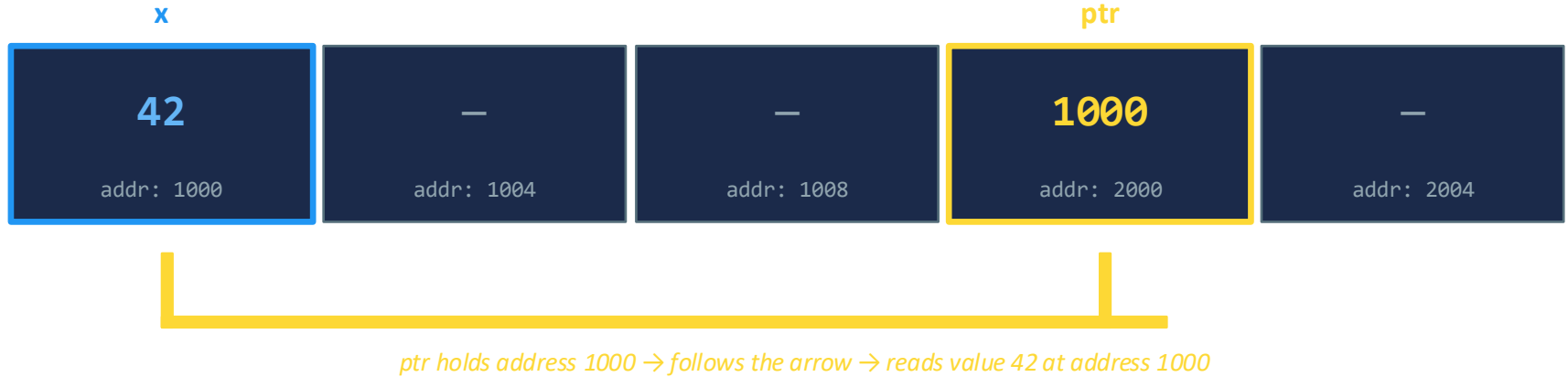
Go to that address and read (or change) the value stored there

Python

```
ctypes.cast(id(x), ctypes.py_object).value
```

```
printf("%d", *ptr); // reads value
```

Pointer Memory Diagram — Step by Step



 **ptr** contains 1000 (an address), not 42 (the value)

 Dereferencing **ptr** means: go to address 1000 and read what's there → 42

 Writing to ***ptr** changes the value at address 1000 → changes **x**

Changing Values Through a Reference

Python (primary)

```
# Integers are immutable – use a list
x = [42]
ptr = x          # ptr references same list

ptr[0] = 99      # change through ptr
print(x[0])      # 99 – x changed too!

# Verify same object
print(id(x) == id(ptr))  # True

# With a dict (also mutable)
data = {"age": 25}
ref = data
ref["age"] = 30
print(data["age"])  # 30 – changed!
```

C (reference only)

```
int x = 42;
int *ptr = &x;

*ptr = 99;          // change via pointer
printf("%d", x);    // 99 – x changed!

// ptr still holds same address
printf("%p", ptr);  // same as &x
```

✓ Mutable objects in Python (lists, dicts, objects) behave like pointers — changes through one reference affect the original.



Practical 2 — Reference Behaviour in Python



15 min

1 Create a = [10] and ptr = a — change ptr[0] = 50, then print a. What happened?

2 Try b = 10 and ref = b — set ref = 50, print b. Different result — why?

3 Print id(a) and id(ptr) — same? Then id(b) and id(ref) after the change — same?

4 Bonus: create student = {"age": 20}, make a reference, change age through it

5 Discussion: what is the difference between mutable and immutable for references?

```
a = [10]
ptr = a
ptr[0] = 50
print(a)           # ?

b = 10
ref = b
ref = 50
print(b)           # ?

print(id(a) == id(ptr))
```

Expected insight: Lists are mutable → references share the same object → a changes. Integers are immutable → ref = 50 creates a new object → b is unchanged.

Arrays and Pointers are Connected

In C, the array name **is** a pointer to the first element. In Python, a list holds **references** to objects at separate addresses.

```
arr = [10, 20, 30, 40]
```

arr[0]

10

addr: 1000

arr[1]

20

addr: 1004

arr[2]

30

addr: 1008

arr[3]

40

addr: 1012

↑ arr points here (address 1000)

Each element is 4 bytes apart →

 Python

```
arr = [10, 20, 30, 40]
print(arr[0])          # 10 - first element
print(id(arr[0]))      # address of first element
print(id(arr[1]))      # different address
```

 C (reference only)

```
int arr[] = {10, 20, 30, 40};
int *ptr = arr;        // = &arr[0]
printf("%d", *ptr);    // 10
printf("%d", *(ptr+1)); // 20
```

Pointer Arithmetic — Moving Through Memory



Python (primary)

```
arr = [10, 20, 30, 40]
offset = 0

# Simulate *(ptr + offset)
while offset < len(arr):
    print(arr[offset])
    offset += 1
# 10 20 30 40
```

C (reference only)

```
int arr[] = {10, 20, 30, 40};
int *ptr = arr;
printf("%d", *(ptr+0)); // 10
printf("%d", *(ptr+1)); // 20
printf("%d", *(ptr+2)); // 30
printf("%d", *(ptr+3)); // 40
```

Key insight: Python's `arr[i]` is equivalent to C's `*(ptr+i)` — same concept, different syntax. In C, `ptr+1` jumps **4 bytes** (`sizeof int`), not 1.



Practical 3 — Array Traversal Using Offsets



15 min

1 Create: scores = [85, 92, 78, 95, 60]

2 Traverse using a manual offset variable — no 'for x in scores' — print scores[offset]

3 Use offset traversal to find the maximum value in the list

4 Reverse-traverse: start at len(scores)-1, count down to 0, print each element

5 Bonus: set ptr = scores, change ptr[2] = 100 — does scores[2] change? Check with id()

```
scores = [85, 92, 78, 95, 60]
offset = 0
```

```
# Traverse (simulate *(ptr+i))
while offset < len(scores):
    print(scores[offset])
    offset += 1
```

```
# Find max using offset
max_val = scores[0]
offset = 1
while offset < len(scores):
    if scores[offset] > max_val:
        max_val = scores[offset]
    offset += 1
```



Bonus hint: ptr = scores makes ptr a reference — id(ptr) == id(scores) → True

Pointer to Pointer (**) — Two Levels of Indirection

A pointer to pointer (**) stores the **address of a pointer** — adding one more level of indirection.



Analogy: Like a signpost that points to another signpost, which points to the destination.

Python

```
x = [42]
ptr = [x]
pptr = [ptr]
print(pptr[0][0][0]) # 42
```



C (reference only)

```
int x=42; int *ptr=&x;
int **pptr=&ptr;
printf("%d", **pptr); // 42
```


Pointer to Pointer — Python vs C

Python (primary)

```
# Simulate with nested lists
x = [42]
ptr = [x]      # ptr "points to" x
pptr = [ptr]    # pptr "points to" ptr

# Read through all levels
print(pptr[0][0][0])    # 42

# Modify through all levels
pptr[0][0][0] = 99
print(x[0])              # 99 - x changed!

# Verify all are same object chain
print(id(ptr[0]) == id(x))    # True
print(id(pptr[0]) == id(ptr)) # True
```

C (reference only)

```
int x = 42;
int *ptr = &x;
int **pptr = &ptr;

printf("%d", x);           // 42 direct
printf("%d", *ptr);        // 42 one level
printf("%d", **pptr);      // 42 two levels

// Change through pptr
**pptr = 99;
printf("%d", x);           // 99

// Address chain
printf("%p", *pptr);       // same as &x
```

Why Use Pointer to Pointer? — Real-World Uses



2D Structures

A list of lists in Python — each inner list is a reference. In C, `int**` manages 2D arrays where each row is a separate pointer.



Modifying References

Pass a container to a function and change what a variable points to inside — useful when you need functions to update pointers.



Command-line Args

C uses `char **argv` = array of strings. Python's `sys.argv` is the direct equivalent — a list of strings passed to your script.



Dynamic Data Structures

Linked lists, trees, graphs — each node holds a pointer to the next. In Python, each object reference is the same idea.



Python equivalent of C's `char **argv`:

```
import sys
print(sys.argv)      # ['script.py', 'arg1', 'arg2']
print(sys.argv[0])   # script name — C: argv[0]
print(sys.argv[1])   # first arg — C: *argv[1]
```



Practical 4 — Nested References in Python



10 min

1 Create `x = [42]`, `ptr = [x]`, `pptr = [ptr]`

2 Read the value by following all three levels: `pptr[0][0][0]`

3 Change `pptr[0][0][0] = 100` — then print `x[0]`. Did it change?

4 Print `id(ptr[0])` and `id(x)` — confirm they are the same object

5 Bonus: create a 2D list `matrix = [[1,2],[3,4]]` — access `matrix[1][0]` using a reference
`row = matrix[1]`

```
x = [42]
ptr = [x]
pptr = [ptr]
```

```
# Step 2 – read
print(pptr[0][0][0])      # 42
```

```
# Step 3 – modify
pptr[0][0][0] = 100
print(x[0])                # ?
```

```
# Step 4 – verify
print(id(ptr[0]) == id(x))
```

```
# Bonus
matrix = [[1, 2], [3, 4]]
row = matrix[1]
print(row[0])              # 3
```

Pointer to a Structure

In Python, objects are already references — assigning `sptr = s1` is equivalent to a pointer in C. Use the `.` operator in both languages.

Python (primary)

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Nimal", 25)
sptr = s1          # reference like a pointer

print(sptr.name)   # "Nimal"
sptr.age = 26
print(s1.age)      # 26 – same object modified!

# Verify same address
print(id(s1) == id(sptr))  # True
```

C (reference only)

```
struct Student {
    char name[50];
    int  age;
};

struct Student s1 = {"Nimal", 25};
struct Student *sptr = &s1;

printf("%s", sptr->name); // "Nimal"
sptr->age = 26;
printf("%d", s1.age);     // 26
```

The . Operator vs the → Operator



Python — always .

Direct or reference — use . either way

```
s1 = Student("Nimal", 25)

# Direct access
print(s1.name)    # "Nimal"
print(s1.age)     # 25

# Reference access
sptr = s1
print(sptr.name)  # "Nimal"
print(sptr.age)   # 25
# Same . operator — Python
# handles it automatically
```



C — . or →

Must choose: direct or pointer?

```
// Direct — use .
struct Student s1 = {"Nimal", 25};
printf("%s", s1.name);    // .
printf("%d", s1.age);     // .

// Pointer — use ->
struct Student *sptr = &s1;
printf("%s", sptr->name); // ->
printf("%d", sptr->age);  // ->
// sptr->age == (*sptr).age
```

Rule — Have the struct itself? Use . | Have a pointer to the struct? Use →



Practical 5 — Object References in Python



10 min

1 Create a Student class with name, age, grade attributes

2 Create instance: `s1 = Student("Kasun", 22, "A")`

3 Create reference: `sptr = s1` — modify `sptr.age = 23`

4 Print `s1.age` — did it change? Print `id(s1) == id(sptr)`

5 Bonus: create a list of 3 Students — iterate using a reference variable and print each

6 Bonus 2: create `sptr2 = Student("Amali", 24, "B")` — does `sptr2` share id with `s1`?

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

s1 = Student("Kasun", 22, "A")
sptr = s1
sptr.age = 23

print(s1.age)           # ?
print(id(s1) == id(sptr)) # ?

# Bonus
students = [
    Student("Nimal", 25, "B"),
    Student("Amali", 23, "A"),
    Student("Kamal", 26, "C"),
]
```

Common Mistakes with References in Python

❌ Expecting a Copy but Getting a Reference

`b = a` for a list gives a reference, not a copy. Change `b` → changes `a` too.

```
a = [1,2,3]
b = a      # reference!
b.append(4)
print(a)   # [1,2,3,4] - oops!
```

❌ Accidentally Mutating Shared Reference

Two variables pointing to the same list — a change through one affects the other.

```
a = [1,2,3]
b = a
b[0] = 99
print(a)  # [99,2,3]
```

🔍 Confusing `is` vs `==`

`==` checks if values are equal. `is` checks if they are the same object (same address).

```
a = [1,2,3]
b = [1,2,3]
print(a == b)  # True - same values
print(a is b)  # False - diff objects
```

✅ When You Need a True Copy

Use `.copy()` for flat lists, `copy.deepcopy()` for nested lists or objects.

```
import copy
a = [[1,2],[3,4]]
b = copy.deepcopy(a)
b[0][0] = 99
print(a[0][0])  # 1 - safe!
```



Golden rule: if you want to share — assign directly. If you want independence — use `.copy()` or `deepcopy()`.

Week 6 Summary

Memory

Every variable has a value and an address. Python: `id()` | C: `&x`

Mutable vs Immutable

Lists/dicts share references. Ints/strings create new objects on reassignment.

Arrays & Pointers

`arr[i]` in Python $\equiv$ `*(ptr+i)` in C. Array name is a pointer to element `[0]`.

Pointers

Variables that store memory addresses. Python vars are references under the hood.

**** Pointer to Pointer**

Nested references — list of lists in Python, `int**` in C. Two levels of indirection.

Objects as Structs

Python class instances are references. Use `.` to access fields — same as C's struct pointer $\rightarrow$



Next: Recursion — functions that call themselves